

Proyecto 3

Chatbot Inteligente con Programación Lógica

Curso: Lenguajes de Programación

Paradigma: Lógico

Semestre: I Semestre 2026

Autores:

Federick Fernández Calderón

Justin Lacayo Picado

Josimar Araya Smith

Repositorio:

<https://github.com/ItsJustStyles/Lenguajes-de-programacion>

Instituto Tecnológico de Costa Rica

2026

Índice general

1 Descripción general	2
2 Enlace al repositorio	2
3 Pasos de instalación	2
3.1 Requisitos	2
3.2 Clonar el repositorio	2
3.3 Ejecutar el chatbot	3
4 Manual de usuario	3
4.1 Inicio y cierre	3
4.2 Consultar conceptos	3
4.3 Solicitar definiciones	3
4.4 Consultar propiedades	4
4.5 Consultar relaciones	4
4.6 Consultar sinónimos	4
4.7 Enseñar nuevos hechos	4
4.8 Olvidar hechos	5
4.9 Patrones de aprendizaje soportados	5
5 Arquitectura lógica	5
5.1 Estructura de archivos	5
5.2 Diagrama de dependencias	5
5.3 Predicados clave por módulo	6
6 Explicación del funcionamiento	7
6.1 Paradigma declarativo	7
6.2 Backtracking en acción	7
6.3 Detección de intención	8
6.4 Aprendizaje dinámico con persistencia	8
6.5 Normalización de entrada	8

1. Descripción general

Este proyecto implementa un **chatbot conversacional** desarrollado íntegramente en el paradigma lógico usando **SWI-Prolog**. El sistema combina una base de conocimiento declarativa, un motor de inferencia automática y la capacidad de aprender nuevos hechos en tiempo de ejecución, persistiéndolos entre sesiones.

Las capacidades principales son:

- Responder preguntas sobre conceptos, definiciones, propiedades y relaciones.
- **Inferir** conocimiento no almacenado explícitamente mediante transitividad taxonómica y herencia de propiedades.
- Reconocer **sinónimos** y variaciones del lenguaje natural (tildes, diferentes formulaciones de una misma pregunta).
- **Aprender** nuevos hechos a partir de comandos en lenguaje natural y recordarlos en sesiones futuras.
- **Olvidar** hechos aprendidos cuando el usuario lo solicita.

2. Enlace al repositorio

```
https://github.com/ItsJustStyles/Lenguajes-de-programacion
```

El código fuente del proyecto se encuentra en la carpeta Proyecto 3/src/.

3. Pasos de instalación

3.1. Requisitos

SWI-Prolog versión 8.0 o superior.

- **Ubuntu/Debian:**

```
sudo apt install swi-prolog
```

- **Windows:** instalador disponible en <https://www.swi-prolog.org/download/stable>
- **macOS:**

```
brew install swi-prolog
```

3.2. Clonar el repositorio

```
git clone https://github.com/ItsJustStyles/Lenguajes-de-programacion.git
cd "Lenguajes-de-programacion/Proyecto 3/src"
```

3.3. Ejecutar el chatbot

```
swipl main.pl
```

El programa inicia automáticamente gracias a la directiva `initialization(iniciar, main)` al final de `main.pl`. No se requieren argumentos adicionales.

4. Manual de usuario

4.1. Inicio y cierre

Al ejecutar `swipl main.pl` el chatbot muestra un mensaje de bienvenida y la lista de comandos disponibles. Para terminar la sesión se escribe cualquiera de: `salir`, `adios`, `chao`, `exit` o `quit`.

```
=====
Chatbot Inteligente - Paradigma Logico
=====
Bot: Hola! Combino inferencia logica con aprendizaje dinamico.

Comandos disponibles:
- "aprende que [X] es [Y]"          -> enseñar un concepto
- "aprende que [X] es un tipo de [Y]" -> enseñar una jerarquia
- "aprende que [X] es parte de [Y]"  -> enseñar composicion
- "olvida que [X] es [Y]"           -> olvidar un hecho
- "salir" / "adios" / "chao"        -> finalizar

Tu: _
```

4.2. Consultar conceptos

```
Tu: que es prolog
Bot: prolog es un lenguaje de programacion logica. Por inferencia,
    tambien es un tipo de: herramienta informatica, lenguaje de
    programacion y software.

Tu: que es python
Bot: python es un lenguaje de programacion de proposito general.
```

4.3. Solicitar definiciones

Las palabras clave `define`, `explica`, `describe`, `explique`, `definicion` activan la intención de definición:

```
Tu: define prolog
Bot: prolog: Prolog es un lenguaje de programacion logica basado en
    la
    logica de predicados de primer orden...

Tu: explica backtracking
Bot: backtracking: El backtracking es el proceso por el cual Prolog,
```

al fallar un objetivo, retrocede al ultimo punto de decision...

Tu: describe recursividad

Bot: recursividad: La recursividad es una tecnica en la que un predicado se define en terminos de si mismo...

4.4. Consultar propiedades

Las palabras clave propiedades, características o tiene activan la búsqueda de propiedades con herencia automática:

Tu: propiedades de perro

Bot: Por inferencia, perro tiene estas propiedades: amamanta crías, crece, muere, nace, respira, se mueve y tiene pelo.

Tu: propiedades del ave

Bot: Por inferencia, ave tiene estas propiedades: crece, muere, nace, pone huevos, respira, se mueve y tiene plumas.

4.5. Consultar relaciones

Tu: relacion entre prolog y ia

Bot: inteligencia artificial se relaciona logicamente con prolog.

Tu: relacion entre cpu y computadora

Bot: Por inferencia de composicion, cpu es parte de computadora.

Tu: relacion entre perro y ser vivo

Bot: Por inferencia taxonomica, perro es un tipo de ser vivo.

4.6. Consultar sinónimos

Tu: sinonimos de computadora

Bot: Estos terminos se pueden usar como sinonimos de computadora: compu, ordenador y pc.

Tu: que es la ia

Bot: [responde sobre inteligencia artificial]

Tu: define el ml

Bot: [responde sobre machine learning]

4.7. Enseñar nuevos hechos

Tu: aprende que typescript es un lenguaje de programacion tipado

Bot: Aprendido! Ahora se que typescript es
lenguaje_de_programacion_tipado
(registrado como es_un).

Tu: aprende que typescript es un tipo de lenguaje de programacion

Bot: Aprendido!

Tu: que es typescript

Bot: typescript es un tipo de: herramienta informatica, lenguaje de programacion, lenguaje de programacion tipado y software.

Los hechos se guardan automáticamente en `learned_facts.pl` y estarán disponibles en la próxima sesión.

4.8. Olvidar hechos

Tu: olvida que typescript es lenguaje de programacion tipado

Bot: Entendido. He olvidado que typescript es `lenguaje_de_programacion_tipado`.

4.9. Patrones de aprendizaje soportados

Patrón de entrada	Tipo registrado
aprende que X es Y	<code>concepto(X, Y)</code>
aprende que X es un Y	<code>es_un(X, Y)</code>
aprende que X es un tipo de Y	<code>es_un(X, Y)</code>
aprende que X es parte de Y	<code>parte_de(X, Y)</code>
aprende que X es sinonimo de Y	<code>sinonimo(X, Y)</code>
aprende que X se relaciona con Y	<code>relacionado(X, Y)</code>

También se aceptan: aprendeme, recuerda, sabias, sabe, ensenate, ensena como variantes de aprende.

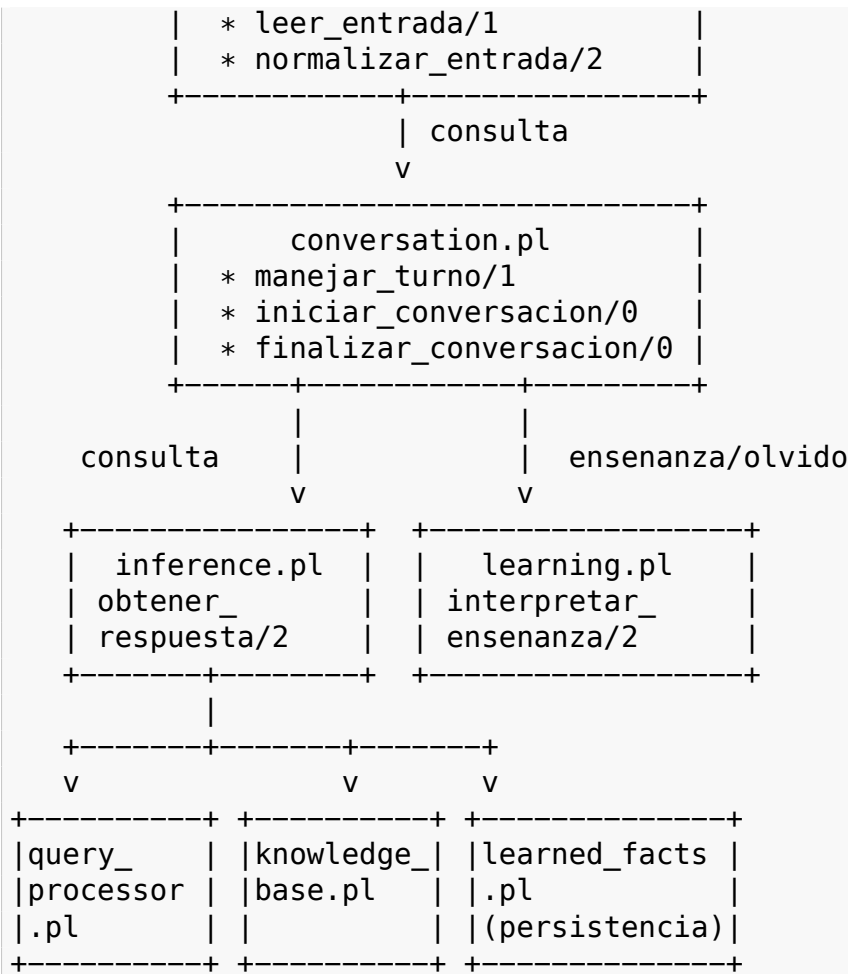
5. Arquitectura lógica

5.1. Estructura de archivos

```
Proyecto 3/
+-- src/
|   +-- main.pl           # Punto de entrada y loop de consola
|   +-- knowledge_base.pl # Base de conocimiento (50+ hechos)
|   +-- inference.pl      # Motor de respuestas e inferencias
|   +-- query_processor.pl # Procesamiento de lenguaje natural
|   +-- learning.pl       # Aprendizaje dinamico y persistencia
|   +-- conversation.pl   # Flujo conversacional y estado
|   +-- learned_facts.pl  # Hechos aprendidos (auto-generado)
+-- DOCUMENTO_TEC.pdf
+-- README.md
```

5.2. Diagrama de dependencias

```
+-----+
|           main.pl           |
| * Loop interactivo          |
+-----+
```



5.3. Predicados clave por módulo

knowledge_base.pl

Predicado	Descripción
concepto/2	Descripción corta de un término
definicion/2	Texto extenso
es_un/2	Jerarquía taxonómica
parte_de/2	Composición estructural
propiedad/2	Característica de una entidad
relacionado/2	Asociación lógica libre
sinonimo/2	Equivalencia de vocabulario
es_un_inferido/2	Transitividad de taxonomía
tiene_propiedad/2	Herencia de propiedades
se_relaciona/2	Simetría de asociaciones
canonico/2	Normalización por sinónimos

inference.pl

Predicado	Descripción
obtener_respuesta/2	Punto de entrada de consultas
obtener_por_intencion/3	Despacho según intención detectada
responder_definicion/2	Respuesta a intención definicion
responder_propiedades/2	Respuesta a intención propiedades
responder_relacion_entre/3	Relación entre dos términos
es_parte_de_inferido/2	Composición transitiva

query_processor.pl

Predicado	Descripción
procesar_pregunta/3	Normaliza + detecta intención y términos
detectar_intencion/2	Clasifica la pregunta en 7 intenciones
detectar_terminos/2	Encuentra términos conocidos en la entrada
limpiar_token/2	Normaliza tildes y elimina puntuación
canonico/2	Resuelve sinónimos a término canónico

6. Explicación del funcionamiento

6.1. Paradigma declarativo

El chatbot es un ejemplo de **programación declarativa**: en lugar de describir *cómo* encontrar una respuesta (con bucles y condiciones), se declaran *qué* hechos son verdaderos y el motor de Prolog deduce las respuestas por unificación y backtracking.

```
% Hecho: el perro es un mamifero
es_un(perro, mamifero).
% Hecho: el mamifero es un animal
es_un(mamifero, animal).

% Regla: inferencia transitiva (no se almacena, se deduce)
es_un_inferido(X, Z) :- es_un(X, Y), es_un_inferido(Y, Z).
```

La consulta `es_un_inferido(perro, animal)` es resuelta por Prolog sin que ningún programador haya escrito explícitamente ese hecho.

6.2. Backtracking en acción

Cuando el sistema busca todas las propiedades de un término, usa `findall/3` que internamente explota el backtracking de Prolog para recopilar todas las soluciones:

```
propiedades_de(Termino, PropiedadesTexto) :-
    soluciones_unicas(Propiedad,
                      tiene_propiedad(Termino, Propiedad),
                      Propiedades),
```



```

maplist(texto_termino, Propiedades, PropiedadesTexto).

soluciones_unicas(Variable, Meta, Soluciones) :-
    findall(Variable, Meta, Lista),
    sort(Lista, Soluciones).

```

Prolog reintenta `tiene_propiedad` con distintas sustituciones de `Propiedad`, recolectando cada una.

6.3. Detección de intención

El módulo `query_processor.pl` clasifica cada entrada en 7 intenciones mediante unificación con listas de palabras clave:

Intención	Palabras clave	Ejemplo
saludo	hola, buenas, saludos	"hola"
definicion	define, explica, describe	"define prolog"
utilidad	sirve, usa, funciona	"para que sirve python"
propiedades	propiedades, características, tiene	"propiedades del perro"
sinonimo	sinonimo, significa, equivale	"sinonimos de ia"
relacion	relacion, relacionado, parte, tipo	"relacion entre X y Y"
consulta_tipo	patrón "que es / que son"	"que es prolog"
consulta_general	(sin palabra clave conocida)	"prolog"

6.4. Aprendizaje dinámico con persistencia

Los predicados `assertz/1` y `retract/1` de Prolog permiten modificar la base de conocimiento en tiempo de ejecución. Para garantizar persistencia entre sesiones, el módulo `learning.pl` escribe cada hecho aprendido en `learned_facts.pl` junto con su registro de tracking:

```

% Contenido de learned_facts.pl tras
% "aprende que robot es una maquina"
:- assertz(concepto(robot, 'una maquina autonoma')).
:- assertz(hecho_aprendido(concepto, robot, 'una maquina autonoma')).

```

Al iniciar una nueva sesión, `inicializar_aprendizaje/0` carga ese archivo y restaura tanto los hechos como su registro, permitiendo que un olvido posterior reescriba el archivo correctamente sin perder hechos previos.

6.5. Normalización de entrada

El pipeline de normalización de texto tiene dos etapas:

1. **main.pl**: convierte a minúsculas y divide por espacios → lista de átomos.
2. **query_processor.pl**: sobre esa lista, `limpiar_token/2` elimina puntuación (`?`, `,`, `!`, `.`) y normaliza caracteres acentuados (`á`→`a`, `é`→`e`, `í`→`i`, `ó`→`o`, `ú`→`u`).

Esto permite que “¿Qué es Prolog?”, “que es prolog” y “Que es Prolog” sean equivalentes para el sistema.